



김정빈 Cloud Engineer

이메일 : vanislab8@gmail.com

전화번호 : 010-2064-9986

GitHub : <https://github.com/SuperSon7>

소개

배포, 운영, 관측 과정의 복잡성을 줄여 **개발자가 비즈니스 로직에 집중할 수 있는 환경**을 만들어왔습니다.

- Terraform IaC 표준화로 인프라 프로비저닝 시간 90% 단축
- Git diff 기반 변경 감지로 불필요한 이미지 빌드를 제거해 Build & Push 시간 30% 단축

시스템을 블랙박스로 사용하기보다 **내부 동작을 직접 확인**하며 인프라를 다뤄온 엔지니어입니다.

- Prometheus 고카디널리티 버킷의 문제를 Alloy relabel·recording rule로 해결
- 컨테이너 런타임을 직접 구현하며 namespace·veth 등 격리·네트워킹 기술의 동작 원리를 검증

글로벌 오픈소스 프로젝트에 코드를 머지하며 리뷰·이슈 기반 협업을 익히고 있습니다.

- openstacksdk에 secret consumer 기능을 머지하며 Gerrit 기반 글로벌 코드 리뷰 사이클을 경험 ([머지코드 링크](#))

프로젝트

독토리 온라인 독서 토론 모임 서비스

3개월 [2025.12.29 ~2026.03.15] | 6명 (백엔드2, 프론트엔드1, Cloud2, AI1)

역할 : **Cloud** 인프라 담당, 모니터링/알림 체계 구축

[Terraform IaC]

- 콘솔을 통한 인프라 관리 중, 빈번한 **휴먼에러와 변경 사항 추적 불가 문제** 해결을 위해 IaC 도입
 - 설계 : base → app → data 3 레이어 구조로 분리. terraform_remote_state 기반 레이어 간 의존성 관리. drift 관리를 위한 scheduled GitHub Action
- 성과 : 전체 인프라 **프로비저닝 4시간 → 25분**으로 단축, 인프라 변경 사항 GitHub 기반 추적관리

[멀티 모듈 빌드 최적화]

- 멀티 모듈에서 일부 변경에도 전체 이미지 빌드 발생, **빌드 시간이 단일 모듈 대비 2배 이상 소요**
 - 원인 : Docker build 내부에서 Gradle 빌드 수행 → layer cache 무효화 전파. GHA Docker cache의 export 비용이 오히려 역효과
 - 해결 : Gradle 빌드를 CI 단계로 분리, Docker Bake 타겟을 모듈별로 분리하고 Git diff 기반 변경 감지로 필요한 타겟만 빌드·푸시.
- 성과 : Build and Push 구간 2m 40s → 1m 52s로 30% 개선, 모듈 간 변경 영향 범위 축소

[Prometheus TSDB 카디널리티 최적화]

- K8s 전환 후 **메트릭 추가에 따른 시계열 폭증**, 디스크 부족으로 모니터링 장애. active series 136,706까지 증가, 알림 rule 평가 부담 가중
 - 해결 : 고카디널리티 메트릭 식별 → 필요하지 않은 대상 제거 → recording rule 도입
- 성과 : 미사용 고카디널리티 시계열 약 34,000개 이상 수집 차단, recording rule을 통한 모니터링 서버 비용 **50% 이상 절감**

[인프라 마이그레이션]

- 실사용자 DB를 단일 EC2(앱+DB 동거)에서 RDS로 전환. **단순 중지 시 mysqldump + import 전체(약 30분) 다운타임.**
 - 해결 : **nginx stream proxy(:3307) → 앱 코드 변경 없이 upstream만 교체.**
super_read_only → **binlog 동기화 확인** → RDS 승격 → **KILL CONNECTION으로 HikariCP 강제 재연결.**
- 성과 : 쓰기 불가 구간 **90% 단축(30분->5초)**. 앱 코드 변경 0. nginx reload로 즉시 롤백 가능.

MiniContainer 컨테이너 런타임 구축

[2026.05 ~] | 개인 프로젝트

목적 : 현대 인프라의 주류인 컨테이너를 구성하는 리눅스 동작 원리 이해

[cgroup 기반 컨테이너 리소스 제한 적용 흐름 개선]

- PID 제한을 설정했지만 컨테이너 내부 프로세스에 **pids.max 제한이 안정적으로 적용되지 않음**
 - **원인 1**: cgroup 설정이 chroot 이후 child 내부에서 실행되면서 host의 실제 /sys/fs/cgroup 이 아니라 rootfs 내부 경로를 바라보고 있음
 - **원인 2**: 리소스 제한은 컨테이너 명령 실행 이후가 아니라, 런타임이 생성한 child process의 host PID에 먼저 적용되어야 이후 fork/exec 되는 프로세스들이 같은 cgroup 제한을 상속
 - **해결**: parent runtime이 child를 Start()로 생성한 직후 host PID를 cgroup.procs에 등록하고, 이후 Wait()하는 구조로 실행 흐름을 분리.
- Docker/Kubernetes의 리소스 제한이 단순 설정값이 아니라, 런타임이 프로세스 생명주기 초기에 커널 cgroup에 프로세스를 배치하는 방식으로 동작한다는 점을 이해

[네트워크 격리 컨테이너에 외부 통신 제공]

- **namespace와 rootfs 격리만**으로는 컨테이너 안에서 독립적인 프로세스 실행은 가능하지만, **외부 네트워크와 통신할 수 없어 실제 서비스 실행 환경으로는 부족**
 - **원인**: 컨테이너는 별도 network namespace를 가지므로 host와 연결할 가상 NIC, 라우팅, NAT 구성이 필요
 - **해결**: veth pair를 생성해 host bridge에 연결 → iptables MASQUERADE 규칙을 추가해 컨테이너 트래픽이 host를 통해 외부로 나갈 수 있게 구성
- 단순히 컨테이너에 IP를 부여하는 것이 아니라, **network namespace 간 veth 연결 + L2 bridge attach + iptables MASQUERADE의 조합으로 외부 통신이 성립**한다는 점을 이해

밥상머리 레시피 커뮤니티

[2025.10~2025.12] | 개인 프로젝트

- AWS 기반 Spring Boot/Express 웹서비스 설계 및 배포
- 100만 MAU 가정 트래픽 산정, ALB·EC2·RDS·Redis·S3 중심 아키텍처 설계
 - <https://url.kr/lucuxi> [아키텍처 설계 문서]
- GitHub Actions CI/CD 구성: Docker 빌드, ECR 푸시, 배포 자동화 경험

VoiceMeSing RVC모형을 이용한 커버곡 생성 서비스

[2024.06~2024.10] | 3인 프로젝트 (AI 파트 모델, 서빙 담당)

- 오픈소스 보이스체인저 AI 모델(RVC) 최적화
- 로컬 GPU 서버 환경 구축 및 오픈소스 모델 서빙

교육사항	카카오테크 부트캠프	클라우드 네이티브	2025년 9월~2026년 3월
	홍익대학교	컴퓨터공학 학사 졸업	2019년 3월~2026년 2월
자격증 및 기타 활동	언어 능력	OPIC(영어)	IM2(2026.04)
	자격증	정보처리기사	2025.06
		AWS DVA	2025.05
	수상	홍익대학교 아이디어톤	대상 (홍익대, 2025.07)
		디자인 엔지니어링 대회	우수상(공학교육혁신센터, 2023.07)
	OpenSource	Openstack	2026.06~